S0.1.2 — Ollama with the chosen models

> **As a Dev, I want Ollama set up with the chosen models, so that LLM and embeddings run locally for free.**

Acceptance criteria (from the backlog):

1. A setup script pulls the LLM (e.g. `gpt-oss:20b` or `qwen3:8b`) and `nomic-embed-text`
2. The API reaches Ollama at a configurable `OLLAMA_BASE_URL`
3. The README notes RAM needs

Size: S (up to half a day). Depends on S0.1.1. The compose stack must be working.

Out of scope, don't build it yet:

- The `/health` check that pings Ollama is S0.2.1.
- The `Embedder` and `LLMClient` classes are S0.2.3.

This story only installs Ollama, gets the models, and proves the containers can reach it. You won't write any backend code.

You'll touch these files:

```
scripts/
└── setup-ollama.ps1     (new)
README.md                (new "Ollama" section)
.env / .env.example      (only if you change the model)
```

Step 1 — Choose the LLM

Ollama runs on the Windows host, not in Docker (decision D10). It shares the machine's RAM with Docker Desktop, VS Code and your browser.

Model	Download	RAM while loaded	Verdict for this project
`qwen3:8b`	~5.2 GB	~6–8 GB	**Use this.** Runs on a 16 GB machine.
`gpt-oss:20b`	~14 GB	~14–16 GB	Needs 32 GB RAM, or a GPU with 16 GB+ VRAM
`nomic-embed-text`	~0.3 GB	< 1 GB	The embedding model (768 dimensions), always needed

This machine has **16 GB RAM** and an **Intel Iris Xe** GPU. Ollama can't use that GPU, so the models run on the **CPU**. On 16 GB, `gpt-oss:20b` with Docker running would swap heavily or fail to load, so stay with `qwen3:8b`. Your `.env.example` already uses it:

```
LLM_MODEL=qwen3:8b
EMBEDDING_MODEL=nomic-embed-text
EMBEDDING_DIM=768
```

Expect answers at a few tokens per second on the CPU. That's slow but fine for development. If it's too slow, `qwen3:4b` (~2.5 GB) is a drop-in swap: change `LLM_MODEL` and re-run the script from step 3.

> `EMBEDDING_DIM` must match the embedding model. It fixes the size of the `VECTOR(n)` column (decision D5). The script in step 3 checks this for you.

Step 2 — Install Ollama on Windows

1. Install it, either with the installer from https://ollama.com/download or with winget:

   ```
   winget install Ollama.Ollama
   ```

2. **Optional: keep the models off drive C.** Models live in `%USERPROFILE%\.ollama\models` by default. To store them on D:, set `OLLAMA_MODELS` **before** pulling anything:

   ```
   setx OLLAMA_MODELS "D:\ollama\models"
   ```

   Then quit Ollama from the system tray and start it again so it picks up the change.

3. **Open a new terminal** so `ollama` is on your PATH, then check it:

   ```
   ollama --version
   Invoke-RestMethod http://localhost:11434/api/version
   ```

   Both should print a version. Ollama runs in the background as a tray app. If the second command fails, start "Ollama" from the Start menu.

Step 3 — Setup script (`scripts/setup-ollama.ps1`)

This is acceptance criterion 1. The script reads the model names from `.env`, so `.env` stays the single source of truth and the script never hard-codes a model. Create `scripts/setup-ollama.ps1` at the repo root:

```
<#
.SYNOPSIS
    Pulls the Ollama models DocQA needs and checks that they work.
.DESCRIPTION
    Reads LLM_MODEL, EMBEDDING_MODEL and EMBEDDING_DIM from .env (or .env.example
```

```
if
    there is no .env yet), pulls both models, checks the embedding size matches
    EMBEDDING_DIM, and loads the LLM once to prove it fits in memory.
    Safe to re-run: models you already have are only updated.
.EXAMPLE
    .\scripts\setup-ollama.ps1
#>
param(
    # The script runs on the host, so it talks to Ollama on localhost
    [string]$OllamaUrl = 'http://localhost:11434'
)

$ErrorActionPreference = 'Stop'

$repoRoot = Split-Path -Parent $PSScriptRoot
$envFile = Join-Path $repoRoot '.env'
if (-not (Test-Path $envFile)) {
    $envFile = Join-Path $repoRoot '.env.example'
}

# Read KEY=value lines; comments and blank lines don't match
$config = @{}
foreach ($line in Get-Content $envFile) {
    if ($line -match '^\s*([A-Z0-9_]+)\s*=\s*(.*?)\s*$') {
        $config[$Matches[1]] = $Matches[2]
    }
}
foreach ($key in 'LLM_MODEL', 'EMBEDDING_MODEL', 'EMBEDDING_DIM') {
    if (-not $config[$key]) { throw "$key is missing from $envFile" }
}
$llm = $config['LLM_MODEL']
$embedder = $config['EMBEDDING_MODEL']
$expectedDim = [int]$config['EMBEDDING_DIM']

if (-not (Get-Command ollama -ErrorAction SilentlyContinue)) {
    throw 'ollama is not on PATH. Install it from https://ollama.com/download,
then open a new terminal.'
}
try {
    $version = Invoke-RestMethod "$OllamaUrl/api/version" -TimeoutSec 5
} catch {
    throw "Ollama is not answering at $OllamaUrl. Start the Ollama app (or run
'ollama serve') and try again."
}
Write-Host "Ollama $($version.version) at $OllamaUrl (settings from $envFile)"

foreach ($model in $llm, $embedder) {
    Write-Host "`nPulling $model ..."
    ollama pull $model
    if ($LASTEXITCODE -ne 0) { throw "ollama pull $model failed" }
}

Write-Host "`nChecking $embedder ..."
$body = @{ model = $embedder; input = 'hello' } | ConvertTo-Json
```

```
$result = Invoke-RestMethod "$OllamaUrl/api/embed" -Method Post -Body $body -
ContentType 'application/json'
$dim = @($result.embeddings[0]).Count
if ($dim -ne $expectedDim) {
    throw "$embedder returns $dim dimensions but EMBEDDING_DIM is $expectedDim.
Make them match in .env."
}
Write-Host "Embeddings OK: $dim dimensions"

# A request with no prompt only loads the model, which proves it fits in memory
Write-Host "`nLoading $llm (the first load can take a minute) ..."
$body = @{ model = $llm } | ConvertTo-Json
Invoke-RestMethod "$OllamaUrl/api/generate" -Method Post -Body $body -ContentType
'application/json' -TimeoutSec 600 | Out-Null
Write-Host "LLM OK: $llm loaded"

Write-Host "`nDone. Try it: ollama run $llm"
```

Why it's written this way:

- **It reads `.env`, not hard-coded names.** If you switch to `qwen3:4b` later, you edit one line in `.env` and re-run the script.
- **It uses `localhost`, not `OLLAMA_BASE_URL`.** The script runs on Windows, where Ollama is `localhost`. `OLLAMA_BASE_URL` (`host.docker.internal`) is the address *from inside the containers*.
- **It checks the embedding dimension.** A mismatch between the model and `EMBEDDING_DIM` would silently break vector search later. Here it fails loudly on day one.
- **It loads the LLM without a prompt.** That proves the model fits in RAM without waiting for `qwen3` to generate a whole answer on the CPU.
- **It's safe to re-run.** `ollama pull` skips layers you already have.

Run it from the repo root:

```
.\scripts\setup-ollama.ps1
```

The first run downloads about 5.5 GB. It should end with `Embeddings OK: 768 dimensions`, `LLM OK: qwen3:8b loaded` and `Done.`

> If you get "*running scripts is disabled on this system*", you skipped the one-time `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` from S0.1.1 step 2.

Try a quick chat. Type `/bye` to exit:

```
ollama run qwen3:8b "In one sentence, what is retrieval-augmented generation?"
```

`qwen3` "thinks" before it answers, so the first reply on a CPU can take a while. That's normal.

Step 4 — Prove the API reaches Ollama

This is acceptance criterion 2. `api` and `worker` already get `OLLAMA_BASE_URL=http://host.docker.internal:11434` from `.env`, and compose maps `host.docker.internal` to the host with `extra_hosts`. You only need to prove it works.

The `python:3.12-slim` image has no `curl`, so use Python's standard library inside the running `api` container:

```
docker compose up -d
docker compose exec api python -c "import os, urllib.request;
print(urllib.request.urlopen(os.environ['OLLAMA_BASE_URL'] + '/api/tags',
timeout=5).read().decode())"
```

It should print JSON listing `qwen3:8b` and `nomic-embed-text`. Run the same command against `worker` too. The worker will call Ollama the most once ingestion arrives in S1.3.1.

> Keep the Python code in **single quotes** inside the double-quoted string. Windows PowerShell 5.1 strips nested double quotes when it passes arguments to `docker`.

Show it's configurable. This is the "configurable" half of the criterion:

1. In `.env`, change the port to a wrong one: `OLLAMA_BASE_URL=http://host.docker.internal:11999`
2. Run `docker compose up -d`. Compose recreates `api` and `worker` because their environment changed.
3. Run the command above again. It should now fail with a connection error.
4. Put the URL back, run `docker compose up -d` again, and confirm it works.

Nothing in the code reads `OLLAMA_BASE_URL` yet. S0.2.1 adds typed config with pydantic-settings, and S0.2.3 builds the Ollama client on top of it. For now, the value only has to reach the containers.

Step 5 — README: Ollama and RAM needs

This is acceptance criterion 3. In `README.md`:

a) Change the Ollama line under **Prerequisites** to:

```
- [Ollama](https://ollama.com/) on the host machine (not in Docker). See [Ollama]
(#ollama) below.
```

b) Add this section after **Getting started**:

```
## Ollama

The LLM and embedding models run in Ollama on the host. Ollama is installed on
Windows, not in Docker. The containers reach it at `OLLAMA_BASE_URL`
(`http://host.docker.internal:11434`).
```

````
Pull the models named in `.env` and check that they work:

```powershell
.\scripts\setup-ollama.ps1
```

Re-run it after you change `LLM_MODEL` or `EMBEDDING_MODEL`.

### RAM needs

| Model | Role | Download | RAM while loaded |
|---|---|---|---|
| `qwen3:8b` (default) | Answers | ~5.2 GB | ~6–8 GB |
| `qwen3:4b` | Lighter alternative | ~2.5 GB | ~3–4 GB |
| `gpt-oss:20b` | Stronger alternative | ~14 GB | ~14–16 GB |
| `nomic-embed-text` | Embeddings (768-d) | ~0.3 GB | < 1 GB |

- **16 GB RAM**: works with `qwen3:8b`. Close other heavy apps while it runs.
- **32 GB RAM**, or a GPU with 16 GB+ VRAM: needed for `gpt-oss:20b`.
- Ollama shares RAM with Docker Desktop (the stack uses about 1–2 GB).
- Without a supported GPU (NVIDIA, or recent AMD), models run on the CPU. That
works, but expect a few tokens per second.
- If you change `EMBEDDING_MODEL`, set `EMBEDDING_DIM` to match. Documents
embedded with the old model must be re-embedded.
````

c) Optionally, add a troubleshooting row:

```
| `api`/`worker` can't reach Ollama | Check `Invoke-RestMethod
http://localhost:11434/api/version` on the host first; see the S0.1.2 guide in
`docs/`. |
```

Done when

- [ ] Ollama is installed and `ollama --version` works
- [ ] `scripts/setup-ollama.ps1` exists, reads models from `.env`, and finishes with `Done.`
- [ ] `ollama list` shows `qwen3:8b` and `nomic-embed-text`
- [ ] `docker compose exec api python -c ...` lists both models from inside the container
- [ ] Changing `OLLAMA_BASE_URL` in `.env` changes what the containers reach
- [ ] The README has the Ollama section with RAM needs
- [ ] `docker compose up` still starts all five services

Suggested commits

```
git add scripts/setup-ollama.ps1
git commit -m "Add Ollama setup script (refs #<S0.1.2 issue>)"
```

```
git add README.md
git commit -m "Document Ollama setup and RAM needs (closes #<S0.1.2 issue>)"
```

Common problems

Symptom	Cause	Fix
`ollama` not recognised	Terminal opened before install	Open a new terminal
Script: "Ollama is not answering"	Tray app not running	Start "Ollama" from the Start menu, or run `ollama serve`
Script: "returns N dimensions but EMBEDDING_DIM is 768"	Wrong embedding model or `EMBEDDING_DIM`	Make them match in `.env` (`nomic-embed-text` gives 768)
LLM load fails or Windows freezes	Not enough free RAM	Close apps, or switch `LLM_MODEL` to `qwen3:4b`
Container check: `Network is unreachable` or `Connection refused`	Ollama isn't running on the host. Python reports the IPv6 attempt first, so the error can say "unreachable".	Check `Invoke-RestMethod http://localhost:11434/api/version` on the host first
Host check works, container check times out	Ollama only listens on `127.0.0.1` and your Docker setup doesn't forward to it	`setx OLLAMA_HOST "0.0.0.0:11434"`, restart Ollama, and allow it through Windows Firewall **on private networks only**
Model downloads filled up C:	Default models folder	Set `OLLAMA_MODELS` (step 2), then pull again. Delete the old `%USERPROFILE%\.ollama\models`
Changed `.env` but containers still see the old URL	Containers not recreated	`docker compose up -d` (not `restart`, which keeps the old environment)

Next: S0.1.3, linting, formatting and CI.